

research

Research: Static Landing Page

Feature: 001-static-landing-page | **Date:** 2026-05-30

Research Tasks

R-001: “Get Started” Button Destination URL

Decision: Use `#install` as the default link target, scrolling to the install snippet section.

Rationale: The spec identifies the destination as an Open Question (OQ-001). Until a documentation URL is confirmed, an in-page anchor provides the best user experience — it takes the visitor directly to the actionable install command, which is the most useful “getting started” step. A bare `#` placeholder would be confusing; a dead external link would be worse. The `#install` anchor is meaningful, functional, and can be trivially updated later.

Alternatives considered: - `#` (bare anchor): Navigates nowhere; poor UX. - External documentation URL: Not yet available; would result in a broken link. - `#features`: Less actionable than `#install`; the features section informs but doesn’t enable.

R-002: Exact Install Command

Decision: Display `npm install -g superspec` as the install snippet.

Rationale: The project name is “SuperSpec” and npm is the most common global-install mechanism for developer tools of this type. The spec assumes this command (OQ-002). The exact package

name and registry must be confirmed before production deployment, but this is the correct default for a working implementation.

Alternatives considered: - `npx superspec`: Avoids global install but implies a run-once workflow; SuperSpec is a persistent CLI tool. - `brew install superspec`: Platform-specific; not available to all developers. - `curl | sh`: Security-conscious developers avoid pipe-to-shell installs.

R-003: Workflow Diagram CSS Approach

Decision: Use a CSS-only step indicator with numbered circles and connecting horizontal lines (flexbox-based).

Rationale: The spec raises OQ-003 about CSS-drawn arrows vs step indicators. CSS arrows (`::after` pseudo-elements with border tricks or rotated elements) have inconsistent rendering across browsers and risk pushing page weight toward the 64 KB constitutional limit due to complex CSS. A step-indicator pattern (numbered circles with horizontal connecting lines) is: - Visually clear and universally understood - Lightweight in CSS (minimal rules needed) - Consistent across browsers - Fully accessible (semantic `<ol>` with visible step numbers)

Alternatives considered: - CSS-drawn arrows (border trick): Browser rendering inconsistencies; more CSS code; risk of 64 KB budget exceedance. - SVG diagram inline: Would work but adds markup weight; harder to make responsive. - CSS Grid arrows with `clip-path`: Not supported in all target browsers; overly complex.

R-004: System Font Stack for Developer Audience

Decision: Use the modern system font stack: `-apple-system`, `BlinkMacSystemFont`, `"Segoe UI"`, `Roboto`, `"Helvetica Neue"`, `Arial`, sans-serif for body text and `"SFMono-Regular"`, `Consolas`, `"Liberation Mono"`, `Menlo`, monospace for code snippets.

Rationale: Constitution Principle I (Zero Runtime Dependencies) prohibits external font CDN links. System fonts render fast, look native on each platform, and are familiar to developers. The monospace stack ensures install command snippets are visually distinct and appropriately technical.

Alternatives considered: - Google Fonts (Inter, JetBrains Mono): Violates constitution — external network dependency. - Web-safe fonts only (Arial, Courier): Adequate but less polished; system font stack includes them as fallbacks.

R-005: CSS Feature Baseline for Target Browsers

Decision: Use Flexbox as the primary layout mechanism. CSS Grid for the features card layout. Both have full support in the last 2 major versions of all target browsers.

Rationale: The constitution limits CSS features to those with baseline support in Chrome, Firefox, Safari, and Edge (last 2 major versions). Flexbox and Grid are universally supported. `clamp()` for responsive font sizing is also baseline-supported and will be used. `forced-colors` media query is supported in all target browsers.

Alternatives considered: - Float-based layout: Unnecessary; flexbox has universal support. - CSS Container Queries: Not yet baseline in Safari; too risky.

R-006: Responsive Breakpoint Strategy

Decision: Mobile-first with two breakpoints: - Base ($\geq 320\text{px}$): Single-column, stacked layout - `@media (min-width: 768px)`: Two-column features grid, horizontal step indicator - `@media (min-width: 1024px)`: Three-column features grid, full-width step indicator

Rationale: Constitution Principle III mandates mobile-first. Three breakpoints cover the spec's viewport range (320px-1920px+) without over-engineering. The 768px and 1024px breakpoints align with common device classes (tablet, desktop).

Alternatives considered: - Four+ breakpoints: Over-engineering for a single static page. - Single breakpoint at 600px: Insufficient — the features grid needs two transitions (stack $\rightarrow$ 2-col $\rightarrow$ 3-col).

R-007: Print Stylesheet Strategy

Decision: Use `@media print` to: hide decorative backgrounds/borders, set body to serif font, remove max-width constraint, force single-column layout, show all content in document order, and preserve the install snippet with a visible border.

Rationale: FR-014 requires an ink-friendly print stylesheet. Decorative elements (hero background, card shadows, step indicator circles) should be suppressed. Content should flow linearly. The install snippet must remain visible (tracked by acceptance criteria).

Alternatives considered: - Separate print CSS file: Violates single-file constraint. - No print styles: Violates FR-014.